

Zabbix 7.4 on AWS EC2

Complete From-Scratch Trainer Runbook

A clean-room build guide starting from the AWS EC2 console: instances, networking, security groups, Ubuntu 24.04, MySQL, Zabbix Server 7.4, Nginx/PHP, Agent 2, host configuration, templates, items, triggers, CPU incident, recovery and troubleshooting.

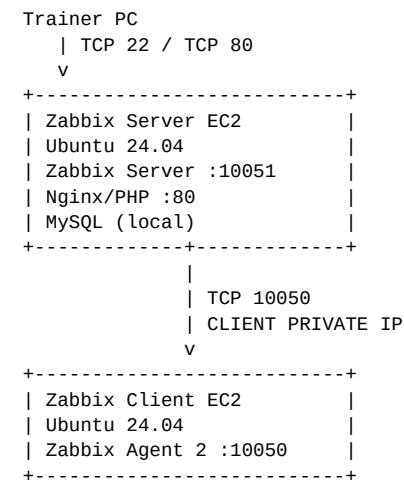
Component	Recommended lab setup
Zabbix Server	Ubuntu 24.04 LTS • t3.medium • 20 GiB gp3
Monitored Client	Ubuntu 24.04 LTS • t3.small • 10 GiB gp3
Database	MySQL 8.x
Frontend	Nginx + PHP
Zabbix	7.4.x from official Zabbix repository
Agent	Zabbix Agent 2
Main demo	CPU >90% sustained for 5 minutes → Problem → Recovery

This version intentionally omits screenshots. It is designed to be followed line-by-line by a trainer rebuilding the environment from an empty AWS setup. Package minor versions may change, so verify them against the official Zabbix 7.4 repository before production use.

Version: September 2026

1. Target architecture

Build two EC2 instances in the same VPC. The server runs Zabbix Server, MySQL and the web frontend. The client runs Zabbix Agent 2. For this lab we use passive checks: the Zabbix Server connects to the client on TCP 10050 using the client's private IP.



Traffic	Source	Destination	Port
Web UI	Trainer PC	Server public IP	TCP 80
SSH	Trainer PC	Each instance public IP	TCP 22
Monitoring	Server private IP	Client private IP	TCP 10050
Server DB	Zabbix Server	Local MySQL	Local socket/3306

For this passive-check lab, the client initiates no connection to TCP 10051. Do not add 10051 just because it is a Zabbix Server port; open only what the selected monitoring architecture actually requires.

2. Plan your values before starting

Variable	Example	Use
Region	ap-south-1	Your chosen AWS region
Server EC2	zabbix-server	EC2 name + Linux hostname
Client EC2	zabbix-client	EC2 name + Linux/Zabbix hostname
DB name	zabbix	MySQL + Zabbix
DB user	zabbix	MySQL + server config
DB password	Your strong password	Use same value everywhere
Server public IP	54.x.x.x	SSH/browser only
Server private IP	172.31.x.x	Agent Server= value
Client private IP	172.31.x.x	Zabbix Agent interface
Your public IP	x.x.x.x/32	SSH/HTTP SG source

3. Create the EC2 key pair

AWS Console → EC2 → Key pairs → Create key pair.

Setting	Value
Name	zabbix-training-key
Type	RSA
Format	.pem for OpenSSH

Download the private key and protect it. On Linux/macOS:

```
chmod 400 zabbix-training-key.pem
```

Example SSH command:

```
ssh -i zabbix-training-key.pem ubuntu@<SERVER_PUBLIC_IP>
```

Do not paste your real private key into the guide, terminal screenshots or classroom material.

4. Create the Zabbix Server security group

EC2 → Security Groups → Create security group → zabbix-server-sg.

Direction	Protocol	Port	Source	Purpose
Inbound	TCP	22	My IP	SSH
Inbound	TCP	80	My IP	Zabbix UI
Outbound	All	All	0.0.0.0/0	Package downloads / outbound traffic

5. Create the Client security group

Create zabbix-client-sg.

Direction	Protocol	Port	Source	Purpose
Inbound	TCP	22	My IP	SSH
Inbound	TCP	10050	zabbix-server-sg	Server → Agent 2
Outbound	All	All	0.0.0.0/0	Default outbound traffic

AWS security groups are instance-level virtual firewalls. Referencing zabbix-server-sg as the source of client TCP/10050 keeps the rule tied to the monitoring server rather than to a changing public IP.

Do not expose MySQL 3306 or Agent 10050 to 0.0.0.0/0. For a short-lived lab, restrict SSH and HTTP to your own IP.

6. Launch the Zabbix Server EC2

EC2 → Instances → Launch instance.

Setting	Recommended value
Name	zabbix-server
AMI	Ubuntu Server 24.04 LTS, 64-bit x86
Instance type	t3.medium
Key pair	zabbix-training-key
VPC	Chosen/default VPC
Subnet	Public subnet
Auto-assign public IP	Enable
Security group	zabbix-server-sg
Root volume	20 GiB gp3

Launch and wait until the instance is Running and both status checks pass.

7. Record the server addresses

Address	Example	Use
Public IPv4	54.x.x.x	SSH + browser
Private IPv4	172.31.x.x	Internal AWS traffic / Agent config

8. Connect

```
ssh -i zabbix-training-key.pem ubuntu@<SERVER_PUBLIC_IP>
```

```
lsb_release -a
hostnamectl
ip -br addr
ip route
```

The public address is for your workstation. Zabbix-to-Agent traffic should use the private address inside the VPC.

9. Launch the monitored Client EC2

Launch a second instance using Ubuntu 24.04 in the same VPC and preferably a public subnet for easy classroom SSH.

Setting	Recommended value
Name	zabbix-client
AMI	Ubuntu Server 24.04 LTS, 64-bit x86
Instance type	t3.small
Key pair	zabbix-training-key
Subnet	Public subnet in same VPC
Auto public IP	Enable for easy SSH
Security group	zabbix-client-sg
Root volume	10 GiB gp3

Record its Public IPv4 and, most importantly, Private IPv4.

```
ssh -i zabbix-training-key.pem ubuntu@<CLIENT_PUBLIC_IP>
```

```
lsb_release -a
hostnamectl
ip -br addr
```

On the client set the hostname:

```
sudo hostnamectl set-hostname zabbix-client
hostnamectl
```

The Zabbix host interface will use the client PRIVATE IP, not its public IP.

10. Prepare the Zabbix Server OS

```
sudo apt update
sudo apt -y upgrade
sudo apt install -y wget curl ca-certificates gnupg2 lsb-release unzip net-tools netcat-openbsd
sudo timedatectl set-timezone Asia/Kolkata
sudo hostnamectl set-hostname zabbix-server
hostnamectl
timedatectl
```

11. Add the official Zabbix 7.4 repository

Zabbix recommends its official repository for supported packages. Ubuntu 24.04 is represented as noble in the repository.

```
cd /tmp
wget https://repo.zabbix.com/zabbix/7.4/release/ubuntu/pool/main/z/zabbix-release/zabbix-release_latest+ubuntu24.04_all.deb
sudo dpkg -i zabbix-release_latest+ubuntu24.04_all.deb
sudo apt update
```

12. Verify available packages

```
apt-cache policy zabbix-server-mysql
apt-cache policy zabbix-agent2
apt-cache policy zabbix-nginx-conf
```

Do not pin a historical minor release unless the training objective requires exact reproducibility. The repository can publish newer 7.4.x packages over time.

13. Install the Zabbix Server stack

```
sudo apt install -y zabbix-server-mysql zabbix-frontend-php zabbix-nginx-conf zabbix-sql-scripts zabbix-agent2 mys
```

This installs the server, PHP frontend, Nginx configuration, database schema package, Agent 2, MySQL and Nginx.

14. Start MySQL

```
sudo systemctl enable --now mysql
sudo systemctl status mysql --no-pager
```

15. Create the Zabbix database and user

```
sudo mysql
```

At the MySQL prompt:

```
CREATE DATABASE zabbix CHARACTER SET utf8mb4 COLLATE utf8mb4_bin;

CREATE USER 'zabbix'@'localhost' IDENTIFIED BY 'ChangeMe-StrongPassword';

GRANT ALL PRIVILEGES ON zabbix.* TO 'zabbix'@'localhost';

SET GLOBAL log_bin_trust_function_creators = 1;

FLUSH PRIVILEGES;
EXIT;

mysql -uzabbix -p -e "SELECT VERSION();"
mysql -uzabbix -p -e "SHOW DATABASES;"
```

16. CRITICAL: Import the Zabbix database schema

Do not start troubleshooting the Zabbix Server until this is complete. A missing schema produces errors such as 'Table zabbix.users doesn't exist' or 'database is not a Zabbix database'.

16.1 Find the schema file installed by the package

```
dpkg -L zabbix-sql-scripts | grep '/mysql/*.sql.gz'
```

In the Ubuntu 24.04 Zabbix 7.4 build used for this lab, the important path is:

```
/usr/share/zabbix/sql-scripts/mysql/server.sql.gz
```

16.2 Import it

```
zcat /usr/share/zabbix/sql-scripts/mysql/server.sql.gz | mysql --default-character-set=utf8mb4 -uzabbix -p zabbix
```

Enter the database password. A successful import normally returns to the shell without a large success message.

16.3 Verify

```
mysql -uzabbix -p -e "USE zabbix; SHOW TABLES LIKE 'users';"  
mysql -uzabbix -p -e "USE zabbix; SHOW TABLES;" | head -20
```

16.4 Why dpkg -L is included

The package name and installed filesystem directory are not necessarily identical. Using `dpkg -L` makes the runbook resilient to package layout surprises instead of relying on an assumed directory.

17. Configure Zabbix Server

Edit the main server configuration:

```
sudo nano /etc/zabbix/zabbix_server.conf
```

Verify/set:

```
DBName=zabbix
DBUser=zabbix
DBPassword=ChangeMe-StrongPassword
```

18. Start and verify

```
sudo systemctl enable --now zabbix-server
sudo systemctl status zabbix-server --no-pager
```

19. If it fails, inspect the log

```
sudo tail -50 /var/log/zabbix/zabbix_server.log
sudo journalctl -u zabbix-server -n 50 --no-pager
```

Do not proceed to the web wizard if zabbix-server is continuously restarting.

20. Verify the database has many tables

```
mysql -uzabbix -p -e "SELECT COUNT(*) AS table_count FROM information_schema.tables WHERE table_schema='zabbix';"
```

A healthy service plus a populated schema is the foundation. Most early Zabbix failures in this lab trace to a database/password/schema mismatch.

21. Configure Nginx for HTTP port 80

Inspect the package-generated configuration:

```
sudo grep -nE 'listen|server_name|root' /etc/zabbix/nginx.conf
```

If it says listen 8080, change it to port 80 for this classroom setup:

```
sudo nano /etc/zabbix/nginx.conf
```

```
# Change:  
listen 8080;
```

```
# To:  
listen 80;
```

Remove the default Nginx site if it competes for port 80:

```
sudo rm -f /etc/nginx/sites-enabled/default
```

22. Test and restart

```
sudo nginx -t  
sudo systemctl enable --now nginx  
sudo systemctl restart nginx  
sudo systemctl status nginx --no-pager
```

23. Verify ports correctly

```
sudo ss -lntp | grep -E ':\s|:10051\s'
```

Do not use `grep ':80'` because it can match `:8080`. The regular expression above checks port 80 as a complete port token.

24. Local HTTP test

```
curl -I http://127.0.0.1
```

A first response such as HTTP 302 redirecting to `setup.php` is normal before the frontend wizard is completed.

25. Complete the Zabbix web wizard

From your trainer PC browse to:

`http://<SERVER_PUBLIC_IP>`

26. Prerequisite screen

Proceed only when the required PHP and web-server checks are green.

27. Database connection screen

Field	Value
Database type	MySQL
Database host	localhost
Database name	zabbix
User	zabbix
Password	Your DB password

28. Zabbix Server details

Field	Value
Host	localhost
Port	10051
Name	AWS Zabbix Training Lab

29. Frontend settings

Field	Value
Language	English
Time zone	(UTC+05:30) Asia/Kolkata
Theme	Blue or your preference
Encryption	HTTP for this disposable lab

30. First login

Complete the wizard. On a fresh default installation, the initial login is typically Admin / zabbix. Change the password immediately after first login.

For a production deployment use HTTPS, a strong administrator password, controlled network access and proper secret management.

31. Install Zabbix Agent 2 on the client

SSH into the client:

```
ssh -i zabbix-training-key.pem ubuntu@<CLIENT_PUBLIC_IP>

sudo apt update
sudo apt -y upgrade
sudo apt install -y wget ca-certificates gnupg2
sudo timedatectl set-timezone Asia/Kolkata

cd /tmp
wget https://repo.zabbix.com/zabbix/7.4/release/ubuntu/pool/main/z/zabbix-release/zabbix-release_latest+ubuntu24.04_all.deb
sudo dpkg -i zabbix-release_latest+ubuntu24.04_all.deb
sudo apt update
sudo apt install -y zabbix-agent2
```

32. Configure Agent 2

```
sudo nano /etc/zabbix/zabbix_agent2.conf
```

Set the server private IP and exact hostname:

```
Server=<SERVER_PRIVATE_IP>
Hostname=Zabbix-client
```

For passive checks, Server= identifies the Zabbix Server allowed to connect to the agent. Hostname must match the host you create in the Zabbix frontend.

33. Start and verify

```
sudo systemctl enable --now zabbix-agent2
sudo systemctl status zabbix-agent2 --no-pager
sudo ss -lntp | grep ':10050\s'
sudo journalctl -u zabbix-agent2 -n 50 --no-pager
```

34. Test Server → Agent connectivity before touching the UI

On the Zabbix Server, test the client's private IP and port 10050:

```
nc -vz <CLIENT_PRIVATE_IP> 10050
```

Expected: a successful TCP connection.

35. If nc fails, troubleshoot in this order

Check	Command/action
Client Agent	sudo systemctl status zabbix-agent2 --no-pager
Client listener	sudo ss -lntp grep ':10050\l'
Agent config	grep -nE '^(Server ServerActive Hostname)= ' /etc/zabbix/zabbix_agent2.conf
Client SG	TCP 10050 source = zabbix-server-sg
Server address	Server= must be Zabbix Server private IP
Network	Both instances must be in same VPC/routable subnets
Agent logs	sudo journalctl -u zabbix-agent2 -n 50 --no-pager

A successful TCP connection proves network reachability. It does not by itself prove that the Agent configuration and Zabbix host definition are correct.

36. Add the client host in the Zabbix UI

Open Data collection → Hosts → Create host.

37. Host definition

Field	Value
Host name	Zabbix-client
Visible name	Zabbix-client
Host group	Linux servers (or your chosen group)
Monitored by	Zabbix server

38. Agent interface

Field	Value
Type	Agent
IP	<CLIENT_PRIVATE_IP>
Port	10050
Connect to	IP

39. Link the template

In Templates, link 'Linux by Zabbix agent'. Do not manually recreate every Linux metric for this basic lab.

Save/Create the host. Wait for the Agent availability indicator to turn green.

If the indicator is red/unknown, stop. Fix communication before teaching items and triggers. This creates a clean troubleshooting habit for students.

40. Teach the Zabbix object model

Object	Meaning	This lab
Host	Machine/device being monitored	Zabbix-client
Template	Reusable monitoring definitions	Linux by Zabbix agent
Item	One collected metric	system.cpu.util
Trigger	Logical expression over item values	CPU >90% for 5m
Problem	Event/state created when trigger is true	High CPU utilization
Graph	Visualization of item history	CPU graph
Action	Automation/notification driven by events	Advanced extension

41. Item vs Trigger

An item answers 'what value should I collect?' A trigger answers 'under what condition should this value be considered a problem?'

Item key:
system.cpu.util

Example trigger:
min(/Zabbix-client/system.cpu.util,5m)>90

42. Why templates matter

A template packages reusable monitoring definitions. Linking a Linux template can provide many CPU, memory, filesystem, network and availability items/triggers without manually creating them host by host.

43. Explain the 5-minute window

The expression uses a time window. A graph can briefly hit 100% and still not generate a Problem because the trigger is asking whether the selected condition remains true over the required window. This is a useful example of alert-noise reduction.

44. Main demo — generate a real CPU Problem

SSH to the client and create sustained CPU load.

```
yes > /dev/null &  
yes > /dev/null &
```

```
jobs  
top
```

Two busy workers are used because on a 2-vCPU instance one process may consume roughly one core, appearing near 50% total CPU.

45. Watch the metric first

In Zabbix, open the client host and CPU graph. Show students that the metric can rise before an alert appears.

46. Wait for the trigger window

The Linux template trigger used in this lab is effectively equivalent to:

```
min(/Zabbix-client/system.cpu.util,5m)>90
```

The trigger therefore needs enough recent data to satisfy its 5-minute condition. The alert is intentionally not an instant reaction to a single spike.

47. Observe the Problem

Open Monitoring → Problems. You should eventually see a warning similar to:

```
Linux: High CPU utilization (over 90% for 5m)
```

48. Stop the load

```
pkill yes
```

Wait for the metric to fall and for the trigger to recover.

If the CPU graph is high but the Problem is not yet visible, do not assume Zabbix is broken. First inspect the trigger expression and its evaluation window.

49. Explain the incident lifecycle

Stage	What happens
Collection	Agent 2 exposes CPU and other host metrics.
Evaluation	Zabbix evaluates trigger expressions as new values arrive.
Problem	A true trigger creates a Problem event.
Acknowledgement	An operator records that the event has been seen/accepted; the condition can remain unhealthy.
Recovery	When the trigger becomes false, Zabbix records recovery.

50. Acknowledgement is not recovery

During the demo, acknowledge the CPU Problem with a comment such as 'Investigating high CPU'. Then show that the event is still a Problem. Only stopping the workload and satisfying the recovery condition makes the monitored condition recover.

51. Suggested student questions

- Why is the client private IP used in the Agent interface?
- Which SG rule allows the monitoring connection?
- What is the difference between an item and a trigger?
- Why does CPU reach 100% before the Problem appears?
- Does acknowledgement make the trigger OK?
- What changes if Agent active checks are used?

52. Troubleshooting — use a bottom-up method

Layer	Question	Test
EC2	Are instances running?	EC2 console
SSH	Can you log in?	ssh -i key ...
Service	Is it active?	systemctl status
Port	Is it listening?	ss -lntp
Network	Can server reach client 10050?	nc -vz ... 10050
Config	Are Server/Hostname/IP correct?	grep config
DB	Does schema exist?	SHOW TABLES
Zabbix host	Correct interface?	Host → Interfaces
Template	Linked?	Host → Templates
Trigger	Correct window?	Trigger expression

53. Zabbix Server restarts repeatedly

```
sudo systemctl status zabbix-server --no-pager
sudo tail -50 /var/log/zabbix/zabbix_server.log
```

If you see 'Table zabbix.users doesn't exist' or 'database is not a Zabbix database', return to the schema section:

```
dpkg -L zabbix-sql-scripts | grep '/mysql/*.sql.gz'
zcat /usr/share/zabbix/sql-scripts/mysql/server.sql.gz | mysql --default-character-set=utf8mb4 -uzabbix -p zabbix
mysql -uzabbix -p -e "USE zabbix; SHOW TABLES LIKE 'users';"
```

54. Browser cannot load Zabbix

```
sudo nginx -t
sudo systemctl status nginx --no-pager
sudo ss -lntp | grep -E ':\s'
curl -I http://127.0.0.1
```

55. Troubleshooting — common live-lab failures

55.1 Nginx is on 8080, but browser uses 80

Inspect `/etc/zabbix/nginx.conf`. If the packaged configuration says `listen 8080`, change it to `listen 80` for this lab, remove the default site if necessary, then run `nginx -t` and restart Nginx.

55.2 Agent availability is red/unknown

```
# Client
sudo systemctl status zabbix-agent2 --no-pager
sudo ss -lntp | grep ':10050\s'
sudo journalctl -u zabbix-agent2 -n 50 --no-pager

# Server
nc -vz <CLIENT_PRIVATE_IP> 10050
```

Then verify:

```
# /etc/zabbix/zabbix_agent2.conf
Server=<SERVER_PRIVATE_IP>
Hostname=Zabbix-client
```

And verify the Zabbix host interface uses the client private IP and port 10050.

55.3 One yes process gives only about 50% CPU

Expected on a 2-vCPU instance. Start two processes for the classroom demo.

55.4 CPU is high but no Problem

Inspect the trigger's time window and confirm the item is receiving data. A sustained 5-minute condition will not become true immediately.

55.5 Port check gives a false match

```
sudo ss -lntp | grep -E ':\s|:10051\s'
```

Use complete-port matching; `grep ':80'` can also match `:8080`.

56. Final validation checklist

Check	Expected result
Server EC2	Running
Client EC2	Running
SSH	Works to both
MySQL	active/running
Zabbix Server	active/running
Nginx	active/running
Agent 2	active/running
Server port 80	Listening
Client port 10050	Listening
Server → Client 10050	nc succeeds
Frontend	Loads on http://<SERVER_PUBLIC_IP>
Client host	Agent availability green
Linux template	Linked
CPU item	Receiving values
CPU graph	Shows data
CPU trigger	Enabled
Incident	Can be reproduced
Recovery	Can be demonstrated

57. Five-minute pre-class smoke test

```
# Server
sudo systemctl is-active mysql
sudo systemctl is-active zabbix-server
sudo systemctl is-active nginx
sudo ss -lntp | grep -E '':80\s'

# Client
sudo systemctl is-active zabbix-agent2
sudo ss -lntp | grep '':10050\s'

# Server → Client
nc -vz <CLIENT_PRIVATE_IP> 10050
```

Run this before students arrive. It catches the most common service, listener and security-group issues before the class starts.

58. Extensions for a longer DevOps/Observability session

Extension	Teaching objective
Memory trigger	Thresholds and sustained conditions
Disk trigger	Filesystem capacity and predictive alerting concepts
Network metrics	Interface monitoring and traffic graphs
Active checks	Compare connection direction with passive checks
Notifications/actions	Turn Problems into operational workflows
Tags	Classify events and route automation
Custom template	Learn reusable monitoring design
SNMP	Introduce network-device monitoring
Proxy	Distributed monitoring across networks
API	Automate host/item/problem operations

59. Passive vs Active checks

Passive	Active
Server connects to Agent	Agent connects to Server
Client needs inbound 10050 from Server	Client needs outbound path to Server 10051
Simple for this lab	Useful where inbound connectivity to monitored hosts is restricted

Choose one model deliberately. Mixing assumptions about 10050 and 10051 is a common classroom troubleshooting mistake.

60. Cleanup and repeatability

60.1 Terminate after class

EC2 → Instances → select zabbix-server and zabbix-client → Instance state → Terminate instance. Remove lab-only security groups and key pairs if they are no longer needed.

60.2 Preserve the lab for future classes

- Capture the final configuration as an AMI or rebuild it with Terraform/CloudFormation.
- Record the Zabbix repository/version used.
- Record security-group rules and instance sizing.
- Keep the CPU incident test as a standard smoke test.
- Never store real production credentials in the trainer document.

61. Suggested 90-minute trainer flow

Time	Activity	Teaching point
0–10	Architecture + SGs	Public/private IP, firewall
10–20	Launch + SSH	EC2 basics
20–35	Install stack	Linux/package management
35–45	DB + schema	Zabbix persistence
45–55	Agent + host	Monitoring path
55–65	Template/items/triggers	Zabbix model
65–75	CPU incident	Metric → trigger → Problem
75–82	Ack/recovery	Incident lifecycle
82–90	Troubleshooting	Bottom-up diagnosis

62. Trainer command sheet

Server

```
sudo apt update && sudo apt -y upgrade
sudo apt install -y wget curl ca-certificates gnupg2 lsb-release net-tools netcat-openbsd
sudo timedatectl set-timezone Asia/Kolkata

cd /tmp
wget https://repo.zabbix.com/zabbix/7.4/release/ubuntu/pool/main/z/zabbix-release/zabbix-release_latest+ubuntu24.04_all.deb
sudo dpkg -i zabbix-release_latest+ubuntu24.04_all.deb
sudo apt update

sudo apt install -y zabbix-server-mysql zabbix-frontend-php zabbix-nginx-conf zabbix-sql-scripts zabbix-agent2 mysql-server

sudo systemctl enable --now mysql

# Create DB/user, then import schema:
dpkg -L zabbix-sql-scripts | grep '/mysql/*.sql.gz'
zcat /usr/share/zabbix/sql-scripts/mysql/server.sql.gz | mysql --default-character-set=utf8mb4 -uzabbix -p zabbix

sudo nano /etc/zabbix/zabbix_server.conf
sudo systemctl enable --now zabbix-server

sudo grep -nE 'listen|server_name|root' /etc/zabbix/nginx.conf
sudo nano /etc/zabbix/nginx.conf
sudo nginx -t
sudo systemctl restart nginx
```

Client

```
sudo apt update && sudo apt -y upgrade
cd /tmp
wget https://repo.zabbix.com/zabbix/7.4/release/ubuntu/pool/main/z/zabbix-release/zabbix-release_latest+ubuntu24.04_all.deb
sudo dpkg -i zabbix-release_latest+ubuntu24.04_all.deb
sudo apt update
sudo apt install -y zabbix-agent2

sudo nano /etc/zabbix/zabbix_agent2.conf
# Server=<SERVER_PRIVATE_IP>
# Hostname=Zabbix-client

sudo systemctl enable --now zabbix-agent2
sudo ss -lntp | grep ':10050\s'
```

CPU demo

```
# Client
yes > /dev/null &
yes > /dev/null &

# Stop
pkill yes

# Server connectivity
nc -vz <CLIENT_PRIVATE_IP> 10050
```

63. Official references and version notes

The installation sequence follows the official Zabbix 7.4 package-based workflow: install the required stack, install Zabbix packages, create/import the database, complete the web interface wizard, then start monitoring. Zabbix's official repository provides Ubuntu packages and the documentation recommends the official repository rather than potentially outdated distribution packages.

Reference	URL
Zabbix 7.4 Installation and first steps	https://www.zabbix.com/documentation/7.4/en/manual/installation
Zabbix 7.4 package installation	https://www.zabbix.com/documentation/7.4/en/manual/installation/install_from_packages
Zabbix 7.4 requirements	https://www.zabbix.com/documentation/7.4/en/manual/installation/requirements
Zabbix 7.4 official repository	https://repo.zabbix.com/zabbix/7.4/
AWS EC2 security groups	https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html
AWS EC2 launch parameters	https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-launch-parameters.html

64. Final mental model

```
AWS provides the compute + network boundary.
|
v
Zabbix Server provides collection + evaluation + event state.
|
+--> MySQL stores Zabbix configuration/history.
|
+--> Nginx/PHP provides the browser UI.
|
+--> Agent 2 exposes metrics from monitored hosts.
|
+--> Templates provide reusable monitoring definitions.
|
+--> Items collect values.
|
+--> Triggers evaluate conditions.
|
+--> Problems represent alert states.
|
+--> Recovery represents return to normal.
```

For the trainer, the key debugging principle is: EC2 → security group → TCP port → service → configuration → Zabbix host/interface → template/item → trigger. Fix one layer at a time.

End of runbook.